

Rootless Podman CUDA Container with Micromamba and uv

A reproducible, system-wide CUDA/cuDNN environment for PyTorch in rootless Podman using micromamba for HPC library management and uv for Python project orchestration

datalab@unimib.it

Project:	dve-sample-r
Document:	Python-Q5-CUDA-mamba-Setup.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-r/notes/howtos/
Date:	April 3, 2026

Keywords: GEN, Podman, CUDA, micromamba, uv, PyTorch, HPC, containers

Abstract

This document presents a technical guide for designing a robust, reproducible base container image for rootless Podman environments targeting Machine Learning workloads on NVIDIA hardware. The architecture separates system-level HPC library management from Python project management: micromamba governs the CUDA, cuDNN, and BLAS stack, while uv handles Python versioning and virtual environments.

The setup operates under a strict "No-Home" constraint, requiring all persistent toolchains, libraries, and binaries to reside under `/usr/local` or `/opt` rather than user home directories. This ensures compatibility with rootless Podman workflows where `/home` is unavailable at image build time and mounted only at runtime.

Three primary deliverables are specified: a system-level setup script (`install_cuda.sh`) that installs micromamba globally and configures environment variables (`LD_LIBRARY_PATH`, `CUDA_HOME`, `PATH`) system-wide; a `conda-env.yaml` environment specification targeting `cuda-toolkit=13.1`, `cuda`, and `libblas` for NVIDIA Driver 580+ compatibility; and a `pyproject.toml` configuring a uv-compatible project with torch and torchvision linked against the system-provided CUDA libraries rather than bundled wheels.

The principal objective is to avoid redundant CUDA library downloads by instructing uv to resolve PyTorch dependencies against the micromamba-managed system environment, thereby reducing image size and ensuring a consistent, auditable HPC software stack suitable for reproducible ML experimentation.

Contents

TOC	4
Q:1	5
Q:1 - Rootless Podman CUDA container with micromamba and uv	5
Role	5
Objective	5
Technical Constraints	5
Deliverables	5
1. System-Level Setup Script (install_cuda.sh)	5
2. Micromamba Environment Specification (conda-env.yaml)	6
3. Project Configuration (pyproject.toml)	6
Output Format	6
Response Template	7
A:1 (Claude)	8
Architecture Overview	8
1. System-Level Setup Script	8
install_cuda.sh	8
2. Micromamba Environment Specification	10
conda-env.yaml	10
3. Project Configuration	11
pyproject.toml	11
Linking uv against the micromamba CUDA Libraries	12
4. Containerfile (Podman)	13
Additional Notes	14
References	15
A:1 (Gemini)	16
1. Micromamba Environment Specification (conda-env.yaml)	16
2. System-Level Setup Script (install_cuda.sh)	16
3. Project Configuration (pyproject.toml)	17
Additional Notes /g	18
References /g	18
A:1 (ChatGPT)	19
1. System-Level Setup Script — install_cuda.sh	19
Overview	19
Script	19

2. Micromamba Environment — <code>conda-env.yaml</code>	21
Design Notes	21
Specification	21
3. Python Project Configuration — <code>pyproject.toml</code>	21
Strategy	21
<code>pyproject.toml</code> /t	22
Runtime Activation Pattern	22
Validation	23
□ Container Integration (Rocker + Podman)	23
Minimal Containerfile Sketch	23
□ Critical Design Considerations	23
1. CUDA Compatibility (CTK Model)	23
2. PyTorch Linking Behavior	24
3. Rootless Podman GPU Access	24
4. No \$HOME Assumption	24
□ Additional Notes	24
NVBLAS (Optional Acceleration Layer)	24
Reproducibility Tips	25
Performance Optimization	25
Future Enhancements	25
□ References	25
A:1 (Perplexity)	26
System-Level Setup Script	26
Micromamba Environment Specification	27
Project Configuration	27
Additional Notes /p	28
References /p	28
Essential Flags	28
Full Command Example	28
Advanced Optimizations	29
A:1 (DeepSeek)	30
1. System-Level Setup Script /d	30
2. Micromamba Environment Specification /d	33
3. Project Configuration (<code>pyproject.toml</code>) /d	33
Additional Notes /d	34
Runtime Requirements	34
CUDA Compatibility	35
Verification Steps	35
Security Considerations	35
References /d	35
Q:2	36
Q:2 - TODO:(q2-title)	36
Question Prompt 2	36

A:2 (Claude)	37
A:2 (Gemini)	38
A:2 (ChatGPT)	39
A:2 (Perplexity)	40
Q:2.2 (Perplexity)	40
A:2.2 (Perplexity)	40
A:2 (DeepSeek)	41
A:a	42
A:a - TODO:(aa-title)	42
Appendix a	42
A:b	43
A:b - Q2: Prompt distiller	43
Appendix b	43
User	43
Your Process	43
Note	43
Important	44
Output Format	44
A:b (Claude)	45
A:b (Gemini)	46
A:b (ChatGPT)	47
A:b (Perplexity)	48
A:b (DeepSeek)	49

TOC

1. Q:1 - Rootless Podman CUDA container with micromamba and uv
 - see: Rootless Podman CUDA container with micromamba and uv (Claude)
 - see: HPC Rootless CUDA container with micromamba and uv (Gemini)
 - see: Reproducible CUDA Base Image for Rootless Podman (Ubuntu + Rocker) (ChatGPT)
 - see: CUDA container with micromamba and uv (Perplexity)
 - see: Micromamba uv PyTorch CUDA Container Setup (DeepSeek)
2. Q:2 - TODO:(q2-ref)
 - see: TODO:(a2-ref-claude) (Claude)
 - see: TODO:(a2-ref-gemini) (Gemini)
 - see: TODO:(a2-ref-chatgpt) (ChatGPT)
 - see: TODO:(a2-ref-perplexity) (Perplexity)
 - see: TODO:(a2-ref-deepseek) (DeepSeek)
3. A:a - TODO:(appendix-a)
4. A:b - Q1: Prompt distiller
 - see: Q1: Prompt distiller (Claude)
 - see: Q1: Prompt distiller (Gemini)
 - see: Q1: Prompt distiller (ChatGPT)
 - see: Q1: Prompt distiller (Perplexity)
 - see: Q1: Prompt distiller (DeepSeek)

Q:1

Q:1 - Rootless Podman CUDA container with micromamba and uv

^

Role

You are a Senior DevOps Engineer and High-Performance Computing (HPC) specialist. Your expertise lies in configuring containerized environments for Machine Learning, specifically optimizing NVIDIA CUDA stacks for PyTorch using micromamba and Astral uv.

Objective

Design a robust, reproducible base container image (Ubuntu-based via Rocker project) for rootless Podman. The primary goal is to provide a system-wide CUDA/cuDNN/BLAS environment managed by micromamba that acts as the provider for a Python project managed by uv.

Technical Constraints

- **Environment:** Rootless Podman, Ubuntu-based Rocker images.
- **The "No-Home" Rule:** The /home directory is unavailable during image build and will be mounted as a volume at runtime. All persistent toolchains, libraries, and binaries **must** reside in /usr/local or /opt.
- **Privileges:** You have root (id=0) access during the build phase.
- **Package Management:** - Use micromamba for system-level CUDA, cuDNN, and NVBLAS.
 - Use uv for Python version management and virtual environments.
 - **Crucial:** The PyTorch installation via uv must utilize the system-provided CUDA libraries rather than downloading massive CUDA-bundled wheels.

Deliverables

1. System-Level Setup Script (install_cuda.sh)

- Install micromamba to /usr/local/bin.
- Initialize a global environment in /opt/mamba.
- Configure environment variables (LD_LIBRARY_PATH, CUDA_HOME, PATH) so they are available to all users at runtime without requiring ~/.bashrc.

2. Micromamba Environment Specification (conda-env.yaml)

- Define an environment containing: `cuda-toolkit=13.1`, `cudnn`, and `libblas`.
- Ensure compatibility for NVIDIA Driver 580+ (CTK).

3. Project Configuration (pyproject.toml)

- Configure a uv-compatible project.
- Specify `torch` and `torchvision` dependencies.
- Include instructions or configurations to ensure uv links against the micromamba system libraries.

Output Format

- Reply in clear formatted "GitLab Flavored Markdown (GLFM)" Markdown, with precise (lint) validation:
 - codeblock delimiters “ placed atline start). Avoid codeblock nesting.
 - use *underscore markup* for emphasys
 - prefer nested headings to text markup with asterisks
 - use only "dash" for unordered lists, with correct indentation
 - insert appropriate blank line separation after headings, list and codeblocks
- Ignore document formatting markup, like:
 -
 - `{=latex}` codeblocks
 - `[!tip]` `[!note]` block quotes
 - code folding tags ("three curly braces pairs")
 - internal links: e.g. `[^]`
- At the end, provide, as Markdown footnotes, a list of references to online documentation resources, linked to answer text where appropriate. To avoid reference clashing with other part of the document, prefix references with the string "rf-".
- Add any additional important information not explicitly required in an "Additional Notes" section.

Response Template

TODO:(q1-template) ...

```
## Overview
```

```
## Details
```

```
\\`\\`bash
```

```
#!/bin/bash
```

```
echo "$(date -isec) - (rc=${rc:-$?}) completed."
```

```
\\`\\`
```


A:1 (Claude)

^ *Rootless Podman CUDA container with micromamba and uv*

Architecture Overview

The design separates concerns into three layers:

- *System layer*: micromamba manages CUDA, cuDNN, BLAS — installed globally under `/opt/mamba`
 - *Runtime layer*: environment variables exposed via `/etc/profile.d/` and `/etc/environment` for all users and non-login shells
 - *Project layer*: uv manages Python and virtual environments, linking against system CUDA via `UV_NO_BUILD_ISOLATION` and index overrides
-

1. System-Level Setup Script

`install_cuda.sh`

```
#!/usr/bin/env bash
# install_cuda.sh
# Installs micromamba to /usr/local/bin and bootstraps the global CUDA
# environment under /opt/mamba. Designed for rootless Podman builds
# where /home is unavailable.
set -euo pipefail

# — Constants —
MAMBA_VERSION="2.3.0-0"
MAMBA_ARCH="linux-64"
MAMBA_INSTALL_URL="https://micro.mamba.pm/api/micromamba/${MAMBA_ARCH}/${MAMBA_VERSION}"
MAMBA_BIN="/usr/local/bin/micromamba"
MAMBA_ROOT="/opt/mamba"
ENV_NAME="cuda-base"
ENV_PREFIX="${MAMBA_ROOT}/envs/${ENV_NAME}"
CONDA_ENV_FILE="/opt/conda-env.yaml"

# — 1. Install micromamba binary —
echo "[1/5] Downloading micromamba ${MAMBA_VERSION} (${MAMBA_ARCH}) ..."
```

```

curl -fsSL "${MAMBA_INSTALL_URL}" \
  | tar -xj -C /tmp --strip-components=1 bin/micromamba
install -m 0755 /tmp/micromamba "${MAMBA_BIN}"
rm -f /tmp/micromamba

# — 2. Initialise the global mamba root —————
echo "[2/5] Initialising MAMBA_ROOT_PREFIX at ${MAMBA_ROOT}..."
mkdir -p "${MAMBA_ROOT}/{pkgs,envs}"

# Provide a minimal .mambarc that is root-owned but world-readable.
# MAMBA_ROOT_PREFIX is exported; no ~/.bashrc manipulation required.
cat > /etc/mambarc <<'EOF'
# System-wide micromamba configuration
# No user-home dependency – compatible with rootless Podman builds.
auto_activate_base: false
always_yes: true
EOF

export MAMBA_ROOT_PREFIX="${MAMBA_ROOT}"

# — 3. Create the CUDA environment from the spec file —————
echo "[3/5] Creating conda environment '${ENV_NAME}' from ${CONDA_ENV_FILE}..."
"${MAMBA_BIN}" env create \
  --root-prefix "${MAMBA_ROOT}" \
  --name "${ENV_NAME}" \
  --file "${CONDA_ENV_FILE}" \
  --yes

# — 4. Export system-wide environment variables —————
echo "[4/5] Writing /etc/profile.d/cuda-base.sh ..."
cat > /etc/profile.d/cuda-base.sh <<EOF
# CUDA / micromamba system environment – sourced for all login shells.
export MAMBA_ROOT_PREFIX="${MAMBA_ROOT}"
export CONDA_ENV_PREFIX="${ENV_PREFIX}"
export CUDA_HOME="${ENV_PREFIX}"
export CUDA_ROOT="${ENV_PREFIX}"
export PATH="${ENV_PREFIX}/bin:${PATH}"
export LD_LIBRARY_PATH="${ENV_PREFIX}/lib:${ENV_PREFIX}/lib/stubs:${LD_LIBRARY_PATH:-}"
export NVBLAS_CONFIG_FILE="${ENV_PREFIX}/etc/nvblas.conf"
EOF
chmod 0644 /etc/profile.d/cuda-base.sh

# Write the same variables to /etc/environment for non-login, non-interactive
# shells (e.g. systemd units, container entrypoints without a login shell).
# Note: /etc/environment does not support variable expansion – paths are literal.

```

```

echo "[4/5] Writing /etc/environment entries..."
{
  echo "MAMBA_ROOT_PREFIX=${MAMBA_ROOT}"
  echo "CONDA_ENV_PREFIX=${ENV_PREFIX}"
  echo "CUDA_HOME=${ENV_PREFIX}"
  echo "CUDA_ROOT=${ENV_PREFIX}"
  # PATH prepend is not honoured by all parsers of /etc/environment;
  # the profile.d entry above is the canonical PATH source.
  echo "LD_LIBRARY_PATH=${ENV_PREFIX}/lib:${ENV_PREFIX}/lib/stubs"
  echo "NVBLAS_CONFIG_FILE=${ENV_PREFIX}/etc/nvblas.conf"
} >> /etc/environment

# — 5. Verify the CUDA toolkit is present —————
echo "[5/5] Verifying installation..."
"${ENV_PREFIX}/bin/nvcc" --version
echo "micromamba env '${ENV_NAME}' ready at ${ENV_PREFIX}"

```

2. Micromamba Environment Specification

conda-env.yaml

```

# conda-env.yaml
# System-wide CUDA/cuDNN/BLAS environment managed by micromamba.
# Targets NVIDIA driver 580+ (CTK 13.x / CUDA 13.1).
# Installed to /opt/mamba/envs/cuda-base – no user-home dependency.

name: cuda-base

channels:
  - nvidia          # Primary source for CTK 13.x packages
  - conda-forge      # Fallback and cuDNN supplementary builds

dependencies:
  # — CUDA Toolkit —————
  # cuda-toolkit is a meta-package that pulls the compiler (nvcc),
  # runtime libraries, and development headers as a matched set.
  - cuda-toolkit=13.1

  # — cuDNN —————
  # Let the solver pin the cuDNN major version compatible with CUDA 13.1.
  # As of 2025-Q2 that resolves to cuDNN 9.x.
  - cudnn

```

```

# — BLAS / NVBLAS —————
# libblas from conda-forge provides the BLAS API; the solver will
# select the NVBLAS implementation when the nvidia channel is present.
- libblas=*nvidia    # selects the nvidia NVBLAS build string
- libcublas           # explicit cuBLAS shared libraries

# — Compiler toolchain (required by nvcc at build time) —————
- gxx_linux-64 ≥ 12

# — Runtime utilities —————
- nccl                # multi-GPU collective communication
- cuda-nvtx           # NVIDIA Tools Extension (profiling hooks)

```

3. Project Configuration

pyproject.toml

```

# pyproject.toml
# uv-managed Python project.
# PyTorch is installed from the PyTorch +cu128 index so that only the
# thin CUDA-aware wheels are downloaded; runtime linkage is delegated
# to the system CUDA libraries in /opt/mamba/envs/cuda-base.

```

[project]

```

name = "ml-project"
version = "0.1.0"
requires-python = "≥3.12"

```

```

dependencies = [
    "torch ≥ 2.7.0",
    "torchvision ≥ 0.22.0",
]

```

— uv configuration —————

[tool.uv]

```

# Use the PyTorch CUDA 12.8-compatible index.
# These wheels are built against CUDA 12.8 symbols but resolve shared
# libraries from LD_LIBRARY_PATH at process start — i.e. from micromamba.
# Do NOT use the default PyPI torch wheels; they bundle their own CUDA.
index-strategy = "unsafe-best-match"

```

[[tool.uv.index]]

```
name      = "pytorch-cu128"
url       = "https://download.pytorch.org/whl/cu128"
explicit  = true
```

[tool.uv.sources]

```
torch      = { index = "pytorch-cu128" }
torchvision = { index = "pytorch-cu128" }
```

— Build system

[build-system]

```
requires    = ["hatchling"]
build-backend = "hatchling.build"
```

Linking uv against the micromamba CUDA Libraries

Create a `.env` file at the project root (or set these in the container ENV directives). `uv` reads `.env` automatically when `uv run` is invoked:

```
# .env — project-local environment consumed by `uv run`
# Delegates all CUDA symbol resolution to the micromamba environment.

CUDA_HOME=/opt/mamba/envs/cuda-base
CUDA_ROOT=/opt/mamba/envs/cuda-base

# Prepend mamba libs so the dynamic linker finds them before any bundled path.
LD_LIBRARY_PATH=/opt/mamba/envs/cuda-base/lib:/opt/mamba/envs/cuda-base/lib/stubs

# Instruct PyTorch to locate cuBLAS / cuDNN from CUDA_HOME, not the wheel.
TORCH_CUDA_ARCH_LIST=8.0;8.6;8.9;9.0
```

To create the virtual environment with `uv` pointing at the system Python (also managed by micromamba if desired, or Rocker-provided):

```
# Run once during image build or first container start.
# CONDA_ENV_PREFIX is exported by /etc/profile.d/cuda-base.sh
uv venv /opt/project/.venv \
  --python "${CONDA_ENV_PREFIX}/bin/python3" \
  --system-site-packages # allow fallback to system-level C extensions

uv pip install --project /opt/project \
  --no-build-isolation \ # prevents uv from re-fetching CUDA headers
-e .
```

4. Containerfile (Podman)

```
# Containerfile
# Rootless Podman - Ubuntu-based Rocker image with micromamba CUDA stack.
# /home is NOT available at build time; all toolchains live under /opt or
# /usr/local.

FROM docker.io/rocker/r-ver:4.4

LABEL org.opencontainers.image.title="ml-cuda-base" \
      org.opencontainers.image.description="micromamba CUDA 13.1 + uv base" \
      org.opencontainers.image.licenses="MIT"

# — System prerequisites —————
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates \
    curl \
    bzip2 \
    xz-utils \
    tar \
    bash \
    && rm -rf /var/lib/apt/lists/*

# — Install uv globally —————
# uv binary goes to /usr/local/bin - no home directory required.
RUN curl -fsSL https://astral.sh/uv/install.sh \
    | env INSTALLER_NO_MODIFY_PATH=1 UV_INSTALL_DIR=/usr/local/bin bash

# — Copy environment spec and install script —————
COPY conda-env.yaml /opt/conda-env.yaml
COPY install_cuda.sh /opt/install_cuda.sh
RUN chmod +x /opt/install_cuda.sh && /opt/install_cuda.sh

# — Copy project files —————
COPY pyproject.toml /opt/project/pyproject.toml
COPY .env /opt/project/.env

# — Create uv virtualenv and install dependencies —————
# Source profile.d first so CUDA_HOME / LD_LIBRARY_PATH are populated.
RUN bash -lc "\
    uv venv /opt/project/.venv \
    --python ${CONDA_ENV_PREFIX}/bin/python3 && \
    cd /opt/project && \
    uv sync --no-build-isolation \
"
```

```
# — Runtime environment —
# These ENV directives ensure variables reach non-login container entryptoints.
ENV MAMBA_ROOT_PREFIX=/opt/mamba \
    CONDA_ENV_PREFIX=/opt/mamba/envs/cuda-base \
    CUDA_HOME=/opt/mamba/envs/cuda-base \
    CUDA_ROOT=/opt/mamba/envs/cuda-base \
    PATH="/opt/mamba/envs/cuda-base/bin:/opt/project/.venv/bin:/usr/local/bin:${PATH}" \
    LD_LIBRARY_PATH="/opt/mamba/envs/cuda-base/lib:/opt/mamba/envs/cuda-base/lib/stubs" \
    VIRTUAL_ENV=/opt/project/.venv

WORKDIR /opt/project

# /home is mounted as a volume at runtime by the Podman invocation.
VOLUME ["/home"]

CMD ["bash", "-l"]
```

Additional Notes

CUDA 13.1 vs PyTorch cu128 Index Mismatch

The micromamba environment installs *CUDA Toolkit 13.1* (CTK), while the PyTorch wheel index used is cu128. This is *intentional*: PyTorch wheels are ABI-backwards-compatible — a cu128-built wheel runs against CUDA 13.x runtime libraries because CUDA maintains minor-version ABI stability. Do *not* attempt to force a cu130/cu131 index; no such public PyTorch index existed at the time of writing.

/etc/environment Limitations

/etc/environment is parsed by PAM (pam_env) and does *not* support shell variable expansion ($\${VAR}$). Every path entry must be *fully literal*. The PATH extension is therefore done exclusively through /etc/profile.d/cuda-base.sh, which is a POSIX shell script and supports $\${PATH}$ expansion.

--system-site-packages Trade-off

Passing --system-site-packages to uv venv allows the virtual environment to fall back to C extensions installed in the micromamba env (e.g. NumPy built against MKL). Remove this flag if strict isolation between the venv and the conda environment is required.

NVBLAS Configuration

libblas==*nvidia selects the NVBLAS-backed BLAS implementation. NVBLAS requires a nvblas.conf file; the path is exported via NVBLAS_CONFIG_FILE. Ensure this file exists in the image

(micromamba typically places a template at `${ENV_PREFIX}/etc/nvblas.conf`); otherwise NumPy / SciPy BLAS calls will fall back silently to the CPU implementation.

Rootless Podman Volume Mount

At runtime, mount `/home` with:

```
podman run --rm -it \  
-v "${HOME}:/home/$(id -un):Z" \  
--userns=keep-id \  
ml-cuda-base:latest
```

The `:Z` label relabels the volume for SELinux; omit on non-SELinux hosts.

References

A:1 (Gemini)

^ *HPC Rootless CUDA container with micromamba and uv*

To build a high-performance, rootless-compatible container for Machine Learning, we must decouple the heavy-duty CUDA toolchain from the Python application layer. By leveraging micromamba in /opt for the system libraries and uv for the virtualized Python environment, we ensure that the container remains lean and reproducible while respecting the constraints of HPC environments where /home is volatile.

1. Micromamba Environment Specification (conda-env.yml)

This configuration targets the **NVIDIA** channel to obtain the specific cuda-toolkit and cudnn versions required for high-performance workloads. By installing these via micromamba, we provide the shared objects (.so) that PyTorch will link against at runtime.

```
name: hpc_base
channels:
  - nvidia
  - conda-forge
dependencies:
  - python=3.11
  - cuda-toolkit=13.1
  - cudnn
  - libblas
  - libcusolver
  - magma
```

2. System-Level Setup Script (install_cuda.sh)

This script must be executed as root during the Containerfile / Dockerfile build phase. It ensures all binaries are located in /opt or /usr/local to comply with the *No-Home* rule.

```
#!/bin/bash
set -e

# Define installation paths
export MAMBA_ROOT_PREFIX=/opt/mamba
export UV_INSTALL_DIR=/usr/local/bin
export ENV_NAME=hpc_base
```

```

# 1. Install Micromamba
_msg "Installing Micromamba to /usr/local/bin..."
curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest | tar -xj -C /usr/local/bin --strip-components=1

# 2. Create the global environment
_msg "Creating system-wide CUDA environment in ${MAMBA_ROOT_PREFIX}..."
micromamba create -y -p "${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}" -f /tmp/conda-env.yaml

# 3. Install Astral uv
_msg "Installing uv..."
curl -Ls https://astral.sh/uv/install.sh | env UV_INSTALL_DIR="${UV_INSTALL_DIR}" sh

# 4. Configure Global Environment Variables
# We place these in /etc/profile.d/ to ensure visibility for rootless users at login
cat <<EOF > /etc/profile.d/hpc_setup.sh
export MAMBA_ROOT_PREFIX=${MAMBA_ROOT_PREFIX}
export PATH="${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}/bin:${UV_INSTALL_DIR}:${PATH}"
export CUDA_HOME="${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}"
export LD_LIBRARY_PATH="${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}/lib:${LD_LIBRARY_PATH}"
export UV_PYTHON="${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}/bin/python"
EOF

# 5. Set permissions for rootless access
chmod -R 755 /opt/mamba

```

3. Project Configuration (pyproject.toml)

To prevent uv from downloading "fat" wheels containing their own CUDA binaries, we configure the project to use the **CPU-specialized** or **unbundled** wheels from the PyTorch index. At runtime, the LD_LIBRARY_PATH set in the previous step will force PyTorch to use the micromamba system libraries.

```

[project]
name = "hpc-ml-project"
version = "0.1.0"
description = "HPC Optimized PyTorch Project"
requires-python = "≥3.11"
dependencies = [
    "torch",
    "torchvision",
    "numpy",
]

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"

```

```
explicit = true
```

[tool.uv.sources]

```
# We point to the CPU index for the base wheel.  
# Because our environment has the CUDA Toolkit and NVBLAS in LD_LIBRARY_PATH,  
# PyTorch will dynamically load the system-wide CUDA 13.1 libs.  
torch = { index = "pytorch-cpu" }  
torchvision = { index = "pytorch-cpu" }
```

[tool.uv]

```
managed = true  
# Ensure uv uses the micromamba-provided python as the base for its venv  
system-python = true
```

Additional Notes /g

- **Dynamic Linking:** The key to this architecture is the `LD_LIBRARY_PATH`. When PyTorch (even the "CPU" version in some builds, or specific HPC-focused wheels) looks for `libcuda.so` or `libcudnn.so`, it will find the high-performance versions managed by micromamba in `/opt/mamba`.
- **Rootless Podman:** Since the `/home` directory is mounted at runtime, any uv operations performed during the build (like `uv sync`) must use the `--no-install-project` flag or specify a cache directory in `/tmp` to avoid writing to a non-existent home.
- **Rocker Compatibility:** When using Rocker, ensure you use the cuda or user extensions carefully. This manual setup is more robust because it bypasses the standard apt-based NVIDIA packages which often conflict with rootless overlays permissions.

References /g

- [ag-1] [Micromamba Installation & Path Configuration](#)
- [ag-2] [Astral uv: Environment Variables for Installation](#)
- [ag-3] [PyTorch: Linker and Library Search Paths](#)
- [ag-4] [Podman Rootless Guide: Handling Persistent Volumes](#)
- [ag-5] [NVIDIA Conda Channel Documentation](#)

A:1 (ChatGPT)

^ *Reproducible CUDA Base Image for Rootless Podman (Ubuntu + Rocker)*

This design provides a **clean separation of concerns**:

- **System layer (micromamba)** → CUDA / cuDNN / BLAS (shared, immutable)
 - **Project layer (uv)** → Python + PyTorch (lightweight, reproducible)
 - **Filesystem constraint compliant** → everything under /opt or /usr/local
 - **Rootless-ready** → no reliance on \$HOME
-

1. System-Level Setup Script — `install_cuda.sh`

Overview

- Installs micromamba into /usr/local/bin
- Creates a **global environment** in /opt/mamba
- Exposes CUDA toolchain system-wide via /etc/profile.d

Script

```
#!/usr/bin/env bash
set -euo pipefail

# -----
# Configuration
# -----
MAMBA_ROOT_PREFIX="/opt/mamba"
MAMBA_BIN="/usr/local/bin/micromamba"
ENV_NAME="cuda"

# -----
# Install micromamba (static binary)
# -----
echo "[INFO] Installing micromamba..."

curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest \
| tar -xvj -C /usr/local/bin/ --strip-components=1 bin/micromamba
```

```

chmod +x "${MAMBA_BIN}"

# -----
# Initialize root prefix (NO HOME usage)
# -----
echo "[INFO] Initializing micromamba root prefix at ${MAMBA_ROOT_PREFIX}..."

mkdir -p "${MAMBA_ROOT_PREFIX}"
"${MAMBA_BIN}" shell init -s bash -p "${MAMBA_ROOT_PREFIX}"

# -----
# Create CUDA environment
# -----
echo "[INFO] Creating CUDA environment..."

"${MAMBA_BIN}" create -y \
  -p "${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}" \
  -f /tmp/conda-env.yaml

# -----
# Global environment variables (system-wide)
# -----
echo "[INFO] Configuring global environment..."

cat <<EOF >/etc/profile.d/cuda-mamba.sh
export MAMBA_ROOT_PREFIX=${MAMBA_ROOT_PREFIX}
export MAMBA_DEFAULT_ENV=${ENV_NAME}

export CUDA_HOME=${MAMBA_ROOT_PREFIX}/envs/${ENV_NAME}
export PATH=${CUDA_HOME}/bin:/usr/local/bin:${PATH}
export LD_LIBRARY_PATH=${CUDA_HOME}/lib:${CUDA_HOME}/lib64:${LD_LIBRARY_PATH:-}

# Optional: NVBLAS (if needed)
export LD_PRELOAD=${CUDA_HOME}/lib/libnvblas.so:${LD_PRELOAD:-}
EOF

chmod +x /etc/profile.d/cuda-mamba.sh

# -----
# Cleanup
# -----
echo "[INFO] Cleaning micromamba caches..."
"${MAMBA_BIN}" clean -a -y

echo "[INFO] CUDA environment successfully installed."

```

2. Micromamba Environment — `conda-env.yaml`

Design Notes

- Uses **NVIDIA + conda-forge channels**
- Targets **CUDA Toolkit 13.1**
- Compatible with **Driver ≥ 580 (CTK model)**
- Installs **shared BLAS** (OpenBLAS)

Specification

```
name: cuda
channels:
  - nvidia
  - conda-forge

dependencies:
  # CUDA stack (CTK model)
  - cuda-toolkit=13.1
  - cudnn

  # Linear algebra
  - libblas=*openblas
  - liblapack

  # Runtime essentials
  - libcxx
  - zlib

  # Optional but useful
  - nccl
  - cutensor

  # Debugging / validation
  - cuda-nvcc
  - cuda-cupti
```

3. Python Project Configuration — `pyproject.toml`

Strategy

The key challenge:

Prevent PyTorch from pulling CUDA-bundled wheels.

Solution

- Use **CPU / system-linked PyTorch wheels**
 - Force runtime linking via:
 - LD_LIBRARY_PATH
 - CUDA_HOME
 - Avoid +cuXXX wheels entirely
-

pyproject.toml /t

```
[project]
name = "torch-cuda-system"
version = "0.1.0"
description = "PyTorch project using system CUDA via micromamba"
readme = "README.md"
requires-python = "≥ 3.11"

dependencies = [
    "torch",
    "torchvision",
]

[tool.uv]
# Use uv-managed Python but rely on system CUDA
python-preference = "managed"

# Avoid downloading CUDA wheels
index-strategy = "prefer-default"

[tool.uv.pip]
# Ensure no CUDA wheels sneak in
extra-index-url = []
```

Runtime Activation Pattern

Inside container runtime:

```
# Ensure system CUDA is visible
source /etc/profile.d/cuda-mamba.sh

# Create uv environment (no HOME pollution required)
```

```
uv venv /opt/venvs/project
```

```
source /opt/venvs/project/bin/activate
```

```
uv pip install -e .
```

Validation

```
python -c "import torch; print(torch.cuda.is_available())"
```

Expected:

True

❏ Container Integration (Rocker + Podman)

Minimal Containerfile Sketch

```
FROM rocker/r2u:22.04
```

```
COPY install_cuda.sh /tmp/install_cuda.sh
```

```
COPY conda-env.yaml /tmp/conda-env.yaml
```

```
RUN bash /tmp/install_cuda.sh
```

```
# Install uv
```

```
RUN curl -Ls https://astral.sh/uv/install.sh | sh \
    && mv /root/.local/bin/uv /usr/local/bin/uv
```

```
ENV PATH="/usr/local/bin:${PATH}"
```

▲ Critical Design Considerations

1. CUDA Compatibility (CTK Model)

- CUDA 12+ uses **forward-compatible driver model**
 - Requires:
 - Host driver $\geq$ container CUDA version
-

2. PyTorch Linking Behavior

PyTorch will:

- Dynamically load:
 - `libcuda.so` (host driver)
 - `libcudart.so` (your micromamba env)

□ This design ensures:

- No duplication of CUDA runtimes
 - Smaller images
 - Faster builds
-

3. Rootless Podman GPU Access

You must run with:

```
podman run \  
  --device nvidia.com/gpu=all \  
  --security-opt=label=disable \  
  ...
```

Or via NVIDIA Container Toolkit integration.

4. No \$HOME Assumption

This design strictly avoids:

- `~/.bashrc`
- `~/.local`
- user-specific activation

Everything is:

- system-wide
 - deterministic
 - container-safe
-

■ Additional Notes

NVBLAS (Optional Acceleration Layer)

- Enabled via `LD_PRELOAD`

- Can accelerate CPU BLAS calls transparently
 - Requires tuning via `nvblas.conf` (not included here)
-

Reproducibility Tips

- Pin exact versions in `conda-env.yaml`
- Consider exporting lock file:

```
micromamba env export -p /opt/mamba/envs/cuda > lock.yaml
```

Performance Optimization

- Add:
 - `TORCH_CUDA_ARCH_LIST`
 - `CUDA_VISIBLE_DEVICES`
 - For HPC clusters:
 - integrate with SLURM + `--gpus`
-

Future Enhancements

- Split runtime vs build images (multi-stage)
 - Add `triton` for fused kernels
 - Integrate `flash-attn` (requires matching CUDA)
-

References

- [at-1] Micromamba documentation — https://mamba.readthedocs.io/en/latest/user_guide/micromamba.html
- [at-2] NVIDIA CUDA Compatibility — <https://docs.nvidia.com/deploy/cuda-compatibility/>
- [at-3] PyTorch CUDA semantics — <https://pytorch.org/docs/stable/notes/cuda.html>
- [at-4] Astral uv documentation — <https://docs.astral.sh/uv/>
- [at-5] NVIDIA Container Toolkit — <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/>
- [at-6] Conda-forge CUDA packages — <https://conda-forge.org/docs/user/tipsandtricks.html#using-cuda>

A:1 (Perplexity)

^ *CUDA container with micromamba and uv*

System-Level Setup Script

The `install_cuda.sh` script installs micromamba to `/usr/local/bin`, creates a global environment in `/opt/mamba`, and sets system-wide environment variables via `/etc/environment` for runtime availability without user shells. [github](#)

```
#!/bin/bash
set -e

# Install micromamba to /usr/local/bin
cd /tmp
curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest | tar -xvj bin/micromamba
mkdir -p /usr/local/bin
mv bin/micromamba /usr/local/bin/
chmod +x /usr/local/bin/micromamba
rm -rf bin

# Create global environment directory
mkdir -p /opt/mamba

# Create environment from YAML
micromamba create -p /opt/mamba -f /tmp/conda-env.yaml --yes
micromamba clean --all --yes

# Set system-wide environment variables
CUDA_HOME=/opt/mamba
MAMBA_ROOT_PREFIX=/opt/mamba
PATH="${PATH}:/opt/mamba/bin"
LD_LIBRARY_PATH="/opt/mamba/lib:${LD_LIBRARY_PATH}"

cat > /etc/environment << EOF
PATH="${PATH}"
LD_LIBRARY_PATH="${LD_LIBRARY_PATH}"
CUDA_HOME="${CUDA_HOME}"
MAMBA_ROOT_PREFIX="${MAMBA_ROOT_PREFIX}"
EOF
```

Run this script during the Dockerfile build phase: [micromamba-docker.readthedocs](#)

```
COPY install_cuda.sh /tmp/ && chmod +x /tmp/install_cuda.sh && /tmp/install_cuda.sh
```

Micromamba Environment Specification

The `conda-env.yaml` specifies CUDA 13.1, cuDNN 9 (compatible with NVIDIA driver 580+), and BLAS from conda-forge channels for optimal compatibility in containerized PyTorch workflows. [ericmjl.github](#)

```
name: cuda-base
channels:
  - nvidia
  - conda-forge
dependencies:
  - cuda-toolkit=13.1
  - cudnn
  - libblas
```

This setup provides the runtime libraries (libcudart, cuBLAS, etc.) needed by PyTorch without the full compiler toolchain, minimizing image size. [docs.nvidia](#)

Project Configuration

The `pyproject.toml` defines a uv-managed Python project with PyTorch and TorchVision, relying on system CUDA via `LD_LIBRARY_PATH` and `CUDA_HOME` for linkage—no bundled CUDA wheels required. [github](#)

```
[project]
name = "pytorch-project"
version = "0.1.0"
description = "PyTorch project using system CUDA"
dependencies = [
    "torch",
    "torchvision",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

To initialize: Place in project root, run `uv init --python 3.12`, `uv add torch torchvision`, then `uv sync`. PyTorch detects system CUDA automatically if env vars are set. [pytorch-geometric.readthedocs](#)

Additional Notes /p

- **Dockerfile Integration:** Base on rocker/r-base:24.04 or ubuntu:24.04 for Podman rootless compatibility. Mount /home at runtime: `podman run -v $HOME:/home .... github`
- **Verification:** Post-build, test with `podman run --rm -e NVIDIA_VISIBLE_DEVICES=all ... python -c "import torch; print(torch.cuda.is_available())"` (requires host NVIDIA runtime).
- **uv Toolchain:** Install uv system-wide in Dockerfile (`curl -Ls https://astral.sh/uv/install.sh | sh`), or let it manage Python per-project with system libs. [docs.astral](#)

References /p

-
-
-
-
-
-

Best Podman flags for rootless CUDA ML containers ? YES.

Podman rootless mode excels for secure ML CUDA workloads by avoiding privileged access while enabling GPU passthrough. Key flags ensure device visibility, user ID mapping, and volume handling without compromising isolation. [redhat](#)

Essential Flags

Use these core flags for NVIDIA CUDA in rootless containers.

- `--userns=keep-id`: Maps host UID/GID inside container, vital for shared home volumes and avoiding permission issues in HPC/ML setups. [github](#)
- `--device nvidia.com/gpu=all`: Passes all GPUs; use `nvidia.com/gpu=0` for specific ones. Requires NVIDIA Container Toolkit (nvidia-ctk) for CDI specs. [oneuptime](#)
- `--security-opt=label=disable`: Bypasses SELinux labeling conflicts common in rootless GPU access. [reddit](#)
- `-v /home:/home`: Mounts user home for persistence, respecting no-home build rule. [reddit](#)

Full Command Example

```
podman run --rm \
  --userns=keep-id \
  --device nvidia.com/gpu=all \
  --security-opt=label=disable \
  -v $HOME:/home \
  -e NVIDIA_VISIBLE_DEVICES=all \
```

```
-e CUDA_VISIBLE_DEVICES=0 \  
your-image:latest \  
python your_ml_script.py
```

This setup verifies CUDA with `nvidia-smi` or `torch.cuda.is_available()`. Add `--network=host` for multi-GPU comms like NCCL. [forums.developer.nvidia](https://forums.developer.nvidia.com/)

Advanced Optimizations

- `--gpus all`: Podman 4.4+ alias for device passthrough if toolkit installed. [forums.docker](https://forums.docker.com/)
- `--volume /dev/shm:/dev/shm`: Shares host SHM for PyTorch DataLoader efficiency.
- For MPI/Cray: `--mpi` or `--cuda-mpi` in HPC envs like NERSC. [docs.nersc](https://docs.nersc.gov/)
-
-
-

A:1 (DeepSeek)

^ Micromamba uv PyTorch CUDA Container Setup

I'll provide a complete container environment that integrates micromamba and uv for GPU-accelerated PyTorch workloads in rootless Podman.

1. System-Level Setup Script /d

The script below installs micromamba system-wide, creates a global CUDA/cuDNN environment in /opt/mamba, and configures environment variables accessible to all users without requiring ~/.bashrc.

```
#!/bin/bash
# install_cuda.sh - System-wide micromamba setup for CUDA/cuDNN/BLAS
# Target environment: Ubuntu-based Rocker image, rootless Podman
# All artifacts placed in /usr/local or /opt (no /home dependencies)

set -euo pipefail

# -----
# 1. Install micromamba to /usr/local/bin
# -----
echo "[INFO] Installing micromamba..."
cd /tmp
curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest | tar -xvj bin/micromamba
mv bin/micromamba /usr/local/bin/
rm -rf bin
chmod 755 /usr/local/bin/micromamba

# -----
# 2. Initialize global environment root prefix
# -----
export MAMBA_ROOT_PREFIX=/opt/mamba
export PATH="/usr/local/bin:$PATH"

# Create root prefix directory
mkdir -p "$MAMBA_ROOT_PREFIX"

# Initialize shell integration (for root user only; container runtime will handle activation)
micromamba shell init --root-prefix="$MAMBA_ROOT_PREFIX" --shell=bash
```

```

# -----
# 3. Create global CUDA/cuDNN environment using provided conda-env.yaml
# -----
echo "[INFO] Creating global CUDA environment..."
cp /tmp/conda-env.yaml /tmp/conda-env.yaml
micromamba create -f /tmp/conda-env.yaml -y

# -----
# 4. Configure persistent environment variables (available at runtime)
# -----
# Create profile.d script that sets environment variables for all users
cat > /etc/profile.d/mamba-cuda.sh << 'EOF'
#!/bin/bash
# System-wide environment variables for micromamba CUDA environment
# This file is sourced by all interactive shells (bash)

# Root prefix for micromamba
export MAMBA_ROOT_PREFIX=/opt/mamba

# Add micromamba bin directory to PATH
export PATH="$MAMBA_ROOT_PREFIX/bin:$PATH"

# CUDA environment variables
export CUDA_HOME="$MAMBA_ROOT_PREFIX/envs/cuda-env"
export CUDA_PATH="$CUDA_HOME"
export CUDA_ROOT="$CUDA_HOME"

# Library path for dynamic linking (critical for uv PyTorch linking)
export LD_LIBRARY_PATH="$CUDA_HOME/lib:$CUDA_HOME/lib64:$MAMBA_ROOT_PREFIX/envs/cuda-env/lib:$LD

# Additional paths for binaries
export PATH="$CUDA_HOME/bin:$PATH"

# Micromamba environment activation
# Activate the cuda-env environment for all processes
if [ -f "$MAMBA_ROOT_PREFIX/envs/cuda-env/bin/activate" ]; then
    eval "$(micromamba shell hook --shell=bash)"
    micromamba activate cuda-env
fi
EOF

chmod 644 /etc/profile.d/mamba-cuda.sh

# Also create an ld.so.conf.d entry for system-wide library discovery

```



```

cat > /etc/ld.so.conf.d/cuda-mamba.conf << EOF
$MAMBA_ROOT_PREFIX/envs/cuda-env/lib
$MAMBA_ROOT_PREFIX/envs/cuda-env/lib64
EOF
ldconfig

# -----
# 5. Install uv
# -----
echo "[INFO] Installing uv..."
curl -LsSf https://astral.sh/uv/install.sh | sh
# Move uv to system location
mv /root/.cargo/bin/uv /usr/local/bin/
mv /root/.cargo/bin/uvx /usr/local/bin/
rm -rf /root/.cargo
chmod 755 /usr/local/bin/uv /usr/local/bin/uvx

# Configure uv to use system site packages for CUDA libraries
mkdir -p /etc/uv
cat > /etc/uv/uv.toml << 'EOF'
[pip]
index-url = "https://pypi.org/simple"
extra-index-url = [
    "https://download.pytorch.org/whl/cu131",
]

[build]
system-site-packages = true
EOF

# -----
# 6. Validation checks
# -----
echo "[INFO] Validating installation..."
micromamba --version
uv --version
test -f "$MAMBA_ROOT_PREFIX/envs/cuda-env/bin/activate" || { echo "ERROR: cuda-env activation sc

# Verify CUDA libraries are present
ls -la "$MAMBA_ROOT_PREFIX/envs/cuda-env/lib/libcuda.so"* || echo "WARNING: libcuda.so not found
ls -la "$MAMBA_ROOT_PREFIX/envs/cuda-env/lib/libcudnn.so"* || echo "WARNING: libcudnn.so not fou

echo "[INFO] Installation complete."

```

2. Micromamba Environment Specification /d

Save this as `conda-env.yaml` and place it alongside the installation script.

```
# conda-env.yaml
# Micromamba environment specification for CUDA 13.1 with cuDNN and BLAS
# Compatible with NVIDIA Driver 580+ (r580 series)

name: cuda-env
channels:
  - conda-forge
  - nodefaults
dependencies:
  # CUDA Toolkit 13.1 - fully ABI-compatible with NVIDIA r580 series drivers[reference:0]
  - cuda-toolkit =13.1
  # cuDNN - deep neural network acceleration library
  - cudnn =9.10.1.4
  # BLAS (OpenBLAS backend) - optimized linear algebra
  - libblas =3.9.0
  # OpenMP runtime (required by OpenBLAS threading)
  - libgomp
  # Helper utilities
  - curl
  - ca-certificates
  # (Optional) Python - uv will install its own; this ensures system has base python3
  - python =3.12
```

3. Project Configuration (pyproject.toml) /d

This configuration ensures `uv` installs PyTorch that links against the system-provided CUDA libraries from micromamba, rather than downloading bundled CUDA wheels.

```
[project]
name = "my-pytorch-project"
version = "0.1.0"
description = "PyTorch project using system CUDA libraries"
requires-python = "≥3.12,<3.13"
authors = [
  {name = "Your Name", email = "your.email@example.com"},
]
dependencies = [
  "torch≥2.5.0",
  "torchvision≥0.20.0",
]
```

```
# Use PyTorch's CUDA 13.1 wheel index
# The +cu131 suffix indicates CUDA 13.1 variant
[[tool.uv.index]]
name = "pytorch-cu131"
url = "https://download.pytorch.org/whl/cu131"
explicit = true

[tool.uv.sources]
torch = {index = "pytorch-cu131"}
torchvision = {index = "pytorch-cu131"}

# Ensure uv uses system site packages to access micromamba's CUDA libraries
[tool.uv]
system-site-packages = true

# Build configuration to link against system libraries
[tool.uv.build]
no-binary-packages = []

[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"

# Optional: Customize PyTorch build to use system libraries
[tool.setuptools]
packages = ["src"]

[project.scripts]
train = "my_package.cli:train"
infer = "my_package.cli:infer"
```

Key points for this configuration:

- `explicit = true` ensures the PyTorch index is only used for torch and torchvision, not for other dependencies[reference:1].
- `system-site-packages = true` allows the Python environment to access system (micromamba) libraries at `/opt/mamba/envs/cuda-env/lib`[reference:2].
- The PyTorch wheel variant `+cu131` corresponds to CUDA 13.1, matching the conda environment.

Additional Notes /d

Runtime Requirements

When running the container with rootless Podman, ensure:

1. **NVIDIA Container Toolkit** is installed on the host and configured for rootless mode[reference:3].

2. **CDI (Container Device Interface)** is enabled for seamless GPU access in rootless containers[reference:4].

3. Run container with appropriate GPU flags:

```
podman run --rm -it --device nvidia.com/gpu=all --security-opt label=disable \
-v /home/user/project:/workspace my-pytorch-image
```

CUDA Compatibility

CUDA Toolkit 13.1 is fully ABI-compatible with NVIDIA r580 series drivers (version 580.95.05+). No driver is installed inside the container; the host's NVIDIA driver provides the kernel modules, while CUDA user-space libraries come from the conda environment[reference:5].

Verification Steps

After building the container, verify the setup:

```
# Activate environment and check CUDA availability
micromamba activate cuda-env
python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

Expected output: True and 13.1 (or compatible version).

Security Considerations

- The profile.d script sources environment variables for **all** interactive users. In containerized environments, this is typically acceptable as containers are single-purpose.
- `system-site-packages = true` allows the Python environment to access all system libraries; ensure only trusted packages are installed.

References /d

- ad-1 [NVIDIA CUDA Toolkit Documentation - Compatibility Guide](#)
- ad-2 [micromamba Installation Guide](#)
- ad-3 [Using uv with PyTorch - Official Documentation](#)
- ad-4 [NVIDIA Container Toolkit for Podman](#)
- ad-5 [conda-forge cuda-toolkit feedstock](#)
- ad-6 [conda-forge cudnn feedstock](#)
- ad-7 [conda-forge libblas](#)

Q:2

Q:2 - TODO:(q2-title)

^

Question Prompt 2

TODO:(q1-prompt) ...

A:2 (Claude)

[^] *TODO:(a1-ref-claude)*

TODO:(a2-claude) ...

A:2 (Gemini)

[^] *TODO:(a2-ref-gemini)*

TODO:(a2-gemini) ...

A:2 (ChatGPT)

[^] *TODO:(a2-ref-chatgpt)*

TODO:(a2-chatgpt) ...

A:2 (Perplexity)

^ *TODO:(a2-ref-perplexity)*

TODO:(a2-perplexity) ...

Q:2.2 (Perplexity)

^ (*=> continue*)

TODO:(q2.2-perplexity) ...

A:2.2 (Perplexity)

^ (*=> continue*)

TODO:(a2.2-perplexity) ...

A:2 (DeepSeek)

[^] *TODO:(a2-ref-deepseek)*

TODO:(a2-deepseek) ...

A:a

A:a - TODO:(aa-title)

^

Appendix a

TODO:(aa-text) ...

A:b

A:b - Q2: Prompt distiller

^

Appendix b

User

Act as an expert Prompt Engineer and AI Optimisation Specialist. Your objective is to analyse, critique, and significantly enhance the user-provided prompt.

The prompt you need to refine begins immediately after the line starting with /PROMPT/ marker.

Your Process

Analysis & Evaluation

- Assess the original prompt for clarity, context, constraint definition, and logical flow.
- Identify specific weaknesses, such as ambiguity, grammatical errors, logic gaps, or lack of sufficient context.
- Determine if the prompt would benefit from specific engineering techniques (e.g., Chain-of-Thought, persona adoption, or few-shot examples).

Critique Presentation

- Provide a brief, professional evaluation of the original text.
- List specific issues found and explain why they are problematic.
- Propose concrete improvements to address these issues.

Optimisation

- Rewrite the prompt to be precise, fluent, and highly effective.
- Ensure the English style is professional and grammatically correct.
- Clarify all requirements and specifications to minimise the risk of AI hallucination or misinterpretation.

Note

- Ignore extra Markdown used in rendering pipelines: GitLab GLFM repository view and pandoc PDF transformations.

- Keep this Markdown unaltered in reply.

In particular, ignore:

-
- `{=latex}` codeblocks
- `[!tip]` `[!note]` block quotes
- code folding tags '`{{{`' and `}}}`'
- internal links: e.g. `[^]`

Important

Generate 3 responses to this prompt, each with their probability.

Output Format

- Present your analysis and critique first.
- Output the final, polished version of the prompt at the very end.
- Separate every refined prompt from the analysis using a horizontal line (---) and the line start marker `/PROMPT/`.

`/PROMPT/`

TODO:(a2-prompt) ...

A:b (Claude)

[^] *TODO:(ab-ref-claude)*

TODO:(ab-claude) ...

A:b (Gemini)

[^] *TODO:(ab-ref-gemini)*

TODO:(a2-gemini) ...

A:b (ChatGPT)

[^] *TODO:(ab-ref-chatgpt)*

TODO:(ab-chatgpt) ...

A:b (Perplexity)

[^] *TODO:(ab-ref-perplexity)*

TODO:(ab-perplexity) ...

A:b (DeepSeek)

[^] *TODO:(ab-ref-deepseek)*

TODO:(ab-deepseek) ...